

Name: Devarjya

Reg No: 240968040

Q1) Write a MPI program using synchronous send. The sender process sends a word to the receiver. The second process receives the word, toggles each letter of the word and sends it back to the first process. Both process use synchronous send operations

Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    char word[100] = "HelloMPI";
    printf("Name: Devarjya, Reg No: 240968040\n");
    MPI_Status status;

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Error: Please run this program with at least 2 processes.\n");
        }
        MPI_Finalize();
        return 0;
    }

    if (rank == 0) {
        printf("Process 0: Original word to send -> %s\n", word);

        MPI_Ssend(word, strlen(word) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
        printf("Process 0: Synchronous send completed.\n");

        MPI_Recv(word, 100, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &status);
        printf("Process 0: Received toggled word back -> %s\n", word);
    } else if (rank == 1) {

        MPI_Recv(word, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process 1: Received word -> %s\n", word);

        for (int i = 0; word[i] != '\0'; i++) {
            if (islower(word[i])) {
                word[i] = toupper(word[i]);
            } else if (isupper(word[i])) {
                word[i] = tolower(word[i]);
            }
        }
    }
}
```

```

    }
}

printf("Process 1: Toggled word -> %s\n", word);

MPI_Ssend(word, strlen(word) + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
printf("Process 1: Synchronous send back completed.\n");
}

MPI_Finalize();
return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 5$ mpicc q1.c -o q1
pcap1@PC-36:~/Documents/240968040/week 5$ mpirun -n 2 ./q1
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Process 0: Original word to send -> HelloMPI
Process 0: Synchronous send completed.
Process 0: Received toggled word back -> hELLOmpi
Process 1: Received word -> HelloMPI
Process 1: Toggled word -> hELLOmpi
Process 1: Synchronous send back completed.

```

Q2) Write a MPI program where the master process (process 0) sends a number to each of the slaves and the slave processes receives the number and prints it. Use standard send.

Code:

```

#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int number_to_send;
    MPI_Status status;
    printf("Name: Devarjya, Reg No: 240968040\n");

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Error: Please run this program with at least 2 processes.\n");
        }
        MPI_Finalize();
        return 0;
    }
}

```

```

if (rank == 0) {
    printf("Master process starting. Total slaves: %d\n", size - 1);

    for (int i = 1; i < size; i++) {
        number_to_send = i * 100; // Example data: 100 for slave 1, 200 for slave 2, etc.

        MPI_Send(&number_to_send, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
        printf("Master sent number %d to Slave (Rank %d)\n", number_to_send, i);
    }

} else {
    int received_number;

    MPI_Recv(&received_number, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

    printf("Slave (Rank %d) successfully received number: %d\n", rank, received_number);
}

MPI_Finalize();
return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 5$ mpicc q2.c -o q2
pcap1@PC-36:~/Documents/240968040/week 5$ mpirun -n 4 ./q2
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Slave (Rank 2) successfully received number: 200
Master process starting. Total slaves: 3
Master sent number 100 to Slave (Rank 1)
Master sent number 200 to Slave (Rank 2)
Master sent number 300 to Slave (Rank 3)
Slave (Rank 1) successfully received number: 100
Slave (Rank 3) successfully received number: 300

```

Q3) Write a MPI program to read N elements of the array in the root process (process 0) where N is equal to the total number of process. The root process sends one value to each of the 22 slaves. Let even ranked process find square of the received element and odd ranked process find cube of received element. Use Buffered send.

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

```

```

int main(int argc, char** argv) {
    int rank, size;
    int *array = NULL;
    MPI_Status status;
    printf("Name: Devarjya, Reg No: 240968040\n");

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int buffer_size = size * (sizeof(int) + MPI_BSEND_OVERHEAD);
    char *bsend_buffer = (char*) malloc(buffer_size);
    MPI_Buffer_attach(bsend_buffer, buffer_size);

    if (rank == 0) {
        array = (int*) malloc(size * sizeof(int));

        printf("Root process: Initializing array with %d elements...\n", size);
        for (int i = 0; i < size; i++) {
            array[i] = i + 1; // Example values: 1, 2, 3, 4, ...
            printf("array[%d] = %d\n", i, array[i]);
        }
        printf("-----\n");

        int root_result = array[0] * array[0];
        printf("Process 0 (Root - Even): Own element = %d, Square = %d\n", array[0], root_result);

        for (int i = 1; i < size; i++) {
            MPI_Bsend(&array[i], 1, MPI_INT, i, 0, MPI_COMM_WORLD);
        }

        free(array);
    } else {
        int received_val;

        MPI_Recv(&received_val, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);

        if (rank % 2 == 0) {
            int square = received_val * received_val;
            printf("Process %d (Even): Received element = %d, Square = %d\n", rank, received_val,
square);
        } else {
            int cube = received_val * received_val * received_val;
            printf("Process %d (Odd): Received element = %d, Cube = %d\n", rank, received_val,
cube);
        }
    }

    int detached_size;
    MPI_Buffer_detach(&bsend_buffer, &detached_size);

```

```

    free(bsend_buffer);

    MPI_Finalize();
    return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 5$ mpicc q3.c -o q3
pcap1@PC-36:~/Documents/240968040/week 5$ mpirun -n 4 ./q3
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Root process: Initializing array with 4 elements...
array[0] = 1
array[1] = 2
array[2] = 3
array[3] = 4
-----
Process 0 (Root - Even): Own element = 1, Square = 1
Process 1 (Odd): Received element = 2, Cube = 8
Process 3 (Odd): Received element = 4, Cube = 64
Process 2 (Even): Received element = 3, Square = 9

```

Q4) Write a MPI program to read an integer value in the root process. Root process sends this value to process1, Process1 sends this value to Process2 and so on. Last process sends the value back to root process. When sending the value each process will first increment the received value by one. Write the program using point to point communication routines.

Code:

```

#include <stdio.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int value = 0;
    MPI_Status status;
    printf("Name: Devarjya, Reg No: 240968040\n");

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Error: Please run this program with at least 2 processes.\n");

```

```

    }
    MPI_Finalize();
    return 0;
}

if (rank == 0) {
    value = 10; // Initial integer value
    printf("Root (Process 0): Initial value = %d\n", value);

    MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("Root (Process 0): Sent value to Process 1\n");

    MPI_Recv(&value, 1, MPI_INT, size - 1, 0, MPI_COMM_WORLD, &status);
    printf("Root (Process 0): Received final value from Process %d = %d\n", size - 1, value);
} else {

    MPI_Recv(&value, 1, MPI_INT, rank - 1, 0, MPI_COMM_WORLD, &status);
    printf("Process %d: Received value = %d\n", rank, value);

    value += 1;
    printf("Process %d: Incremented value to = %d\n", rank, value);

    int dest = (rank + 1) % size;

    MPI_Send(&value, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
    printf("Process %d: Sent value to Process %d\n", rank, dest);
}

MPI_Finalize();
return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 5$ mpicc q4.c -o q4
pcap1@PC-36:~/Documents/240968040/week 5$ mpirun -n 4 ./q4
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Root (Process 0): Initial value = 10
Root (Process 0): Sent value to Process 1
Process 1: Received value = 10
Process 1: Incremented value to = 11
Process 1: Sent value to Process 2
Process 2: Received value = 11
Process 2: Incremented value to = 12
Process 2: Sent value to Process 3
Process 3: Received value = 12
Process 3: Incremented value to = 13
Root (Process 0): Received final value from Process 3 = 13
Process 3: Sent value to Process 0

```

Q5) Write a MPI program to accept the input from .txt file by root process, and extract the text in each line. Send each statement to separate process and count the total number of word by each process. Display the word count by each process and send back the count to root process. Finally display the total word count by the root process

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;
    printf("Name: Devarjya, Reg No: 240968040\n");

    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) {
            printf("Error: Please run this program with at least 2 processes.\n");
        }
        MPI_Finalize();
        return 0;
    }
}

```

```

if (rank == 0) {
    FILE *file = fopen("input.txt", "r");
    if (file == NULL) {
        printf("Error: Could not open input.txt\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    char line[256];
    int active_slaves = size - 1;
    int line_index = 1;

    printf("Root Process: Reading lines from input.txt and distributing...\n");
    printf("-----\n");

    for (int i = 1; i < size; i++) {
        if (fgets(line, sizeof(line), file) != NULL) {
            line[strcspn(line, "\n")] = 0;

            MPI_Send(line, strlen(line) + 1, MPI_CHAR, i, 0, MPI_COMM_WORLD);
        } else {
            // If there are fewer lines in the file than slave processes, send an empty string
            char *empty = "";
            MPI_Send(empty, 1, MPI_CHAR, i, 0, MPI_COMM_WORLD);
        }
    }
    fclose(file);

    int total_word_count = 0;
    for (int i = 1; i < size; i++) {
        int slave_word_count = 0;
        MPI_Recv(&slave_word_count, 1, MPI_INT, i, 0, MPI_COMM_WORLD, &status);
        total_word_count += slave_word_count;
    }

    printf("-----\n");
    printf("Root Process: Total word count across all lines = %d\n", total_word_count);
} else {
    char received_line[256];

    MPI_Recv(received_line, 256, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &status);

    int word_count = 0;
    int in_word = 0;

    for (int i = 0; received_line[i] != '\0'; i++) {
        if (isspace(received_line[i])) {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            word_count++;
        }
    }
}

```



```

}

printf("Process %d: Line = \"%s\" -> Word Count = %d\n", rank, received_line, word_count);

MPI_Send(&word_count, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 5$ mpicc q5.c -o q5
pcap1@PC-36:~/Documents/240968040/week 5$ mpirun -n 4 ./q5
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Name: Devarjya, Reg No: 240968040
Root Process: Reading lines from input.txt and distributing...
-----
Process 1: Line = "Hello world from MPI" -> Word Count = 4
Process 3: Line = "Learning C programming step by step" -> Word Count = 6
Process 2: Line = "Parallel computing is fun and powerful" -> Word Count = 6
-----
Root Process: Total word count across all lines = 16
pcap1@PC-36:~/Documents/240968040/week 5$ cat input.txt
Hello world from MPI
Parallel computing is fun and powerful
Learning C programming step by step

```